ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 08

Condicionales: tomar decisiones

- Qué es una condición
- if: hacer algo solo si…
- if…else: uno de dos caminos
- elseif: varias opciones (y por qué el orden importa)
- Condiciones compuestas: &&, || y !
- Pensar antes de programar y no anidar
- La trampa del igual solitario
- switch: la forma clásica de elegir
- match: la forma moderna
- El operador ternario
- Los operadores del PHP moderno: ?:, ??, ??= y ?→
- La nave espacial: ⟺
- Condicionales dentro del HTML
- ¿Qué uso en cada caso?



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Hasta ahora nuestros programas eran muy obedientes: ejecutaban la línea 1, luego la 2, luego la 3... y siempre lo mismo. Pero un programa que sirva para algo tiene que **decidir**: si el usuario es mayor de edad, si la contraseña es correcta, si queda stock, si hoy es miércoles y la entrada del cine es más barata...

Pensad en el **portero de una discoteca**. Tiene unas normas en la cabeza: si eres mayor de 18, pasas; si no, a casa; y si traes entrada VIP, quizá hay una excepción. Eso es exactamente un condicional. Hoy dedicamos la clase entera a ellos: desde el `if` de toda la vida hasta los operadores que ha traído el PHP moderno (`match` , `?:` , `??` , `??=` y `?→`).

•

👌 Cómo vamos a trabajar



1

📘 Qué es una condición



2

👍 if: hacer algo solo si...



3

👌 if...else: uno de dos caminos



4

🚀 elseif: varias opciones (y por qué el orden importa)



5

🔍 Condiciones compuestas: &&, || y !



6

🤔 Pensar antes de programar y no anidar



7

⚠️ La trampa del igual solitario



8

📘 switch: la forma clásica de elegir



9

🚀 match: la forma moderna



10

😮 El operador ternario



11

🔍 Los operadores del PHP moderno: ?:, ??, ??= y ?→



12

👌 La nave espacial: ⇔



13

👍 Condicionales dentro del HTML



14

🤔 ¿Qué uso en cada caso?



15

🚀 **Ejercicios**



16

👋 **Lo que tenéis que llevaros de hoy**





CLASE 08

👌 Cómo vamos a trabajar

Seguimos en el proyecto `php-paso-a-paso`. Cread la carpeta `clase08` y abrid cada archivo en `http://php-paso-a-paso.test/clase08/nombre-del-archivo.php`.

Hoy hay **21 ejemplos** en `ejemplos/`. No os limitéis a leerlos: escribidlos, cambiad los valores de las variables y comprobad que el resultado es el que esperabais. Con los condicionales, **cambiar los datos y volver a probar** es la mejor forma de aprender.



CLASE 08 · PUNTO 1

📘 Qué es una condición

Una **condición** es una pregunta que solo tiene dos respuestas: **verdadero** (`true`) o **falso** (`false`). En la clase 7 vimos los operadores que hacen esas preguntas:

Operador	Pregunta	Ejemplo	Resultado
`===`	¿Son iguales (y del mismo tipo)?	`5 === 5`	`true`
`!==`	¿Son distintos?	`5 !== 3`	`true`
`>` / `<`	¿Mayor que? / ¿Menor que?	`7 > 10`	`false`
`>=` / `<=`	¿Mayor o igual? / ¿Menor o igual?	`18 >= 18`	`true`

Y recordad que algunos valores **cuentan como falsos** aunque no sean `false`: el `0`, el `0.0`, el texto vacío `""`, el texto `"0"`, el array vacío `[]` y `null`. Todo lo demás cuenta como verdadero. Lo vais a necesitar hoy.



CLASE 08 · PUNTO 2

👍 if: hacer algo solo si...

La forma más sencilla: **si** se cumple la condición, se ejecuta lo que hay entre llaves; si no, PHP se lo salta y sigue [1].

01-if-simple.php — php-paso-a-paso/clase08 — Visual Studio Code

01-if-simple.php

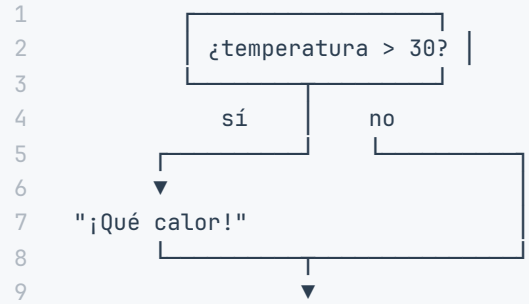
```php
<?php
$temperatura = 31;

if ($temperatura > 30) {
    echo "🥵 ¡Qué calor! Bebed agua.<br>";
}

echo "Temperatura actual: $temperatura °C";
```

📄 Ejemplo: `01-if-simple.php`. Cambiad la temperatura a 25: el aviso desaparece, pero la última línea sale siempre, porque está **fuera** del `if`.

Esquema





💡 Las llaves y la sangría de 4 espacios no son decoración: os dicen de un vistazo **qué depende** del `if`. Ponedlas siempre, aunque dentro haya una sola línea.



CLASE 08 · PUNTO 3

👌 if...else: uno de dos caminos

Con `else` («si no») damos un camino alternativo. Se ejecuta **siempre uno** de los dos, **nunca los dos**:

02-if-else.php — php-paso-a-paso/clase08 — Visual Studio Code

02-if-else.php

```php
<?php
$numero = 7;

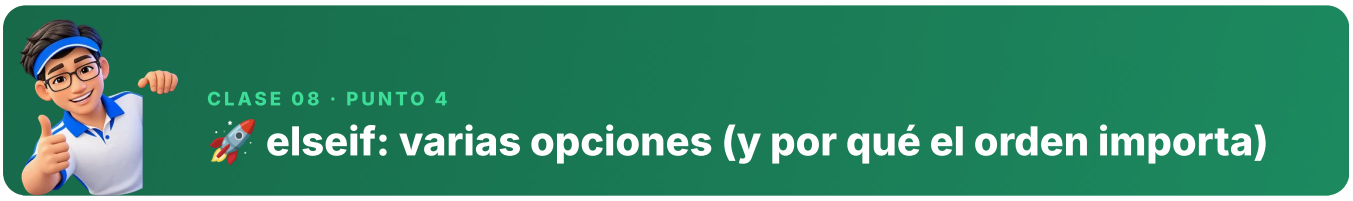
if ($numero % 2 == 0) {
    $tipo = "par";
} else {
    $tipo = "impar";
}

echo "El $numero es $tipo";
```

📄 Ejemplo: `02-if-else.php`

¿Recordáis el operador `%` (resto de la división) de la clase 7? Un número es par si al dividirlo entre 2 el resto es 0. Lo vais a usar muchísimo.

Fijaos también en el estilo: dentro del `if` **no** escribimos el `echo`; solo **calculamos** `$tipo`. La salida va al final. Así el programa queda ordenado: primero se decide y después se muestra.



CLASE 08 · PUNTO 4

🚀 elseif: varias opciones (y por qué el orden importa)

Cuando hay más de dos caminos, encadenamos condiciones con `elseif` («si no, si...»):

03-elseif-notas.php — php-paso-a-paso/clase08 — Visual Studio Code

03-elseif-notas.php

```php
<?php
$nota = 7.5;

if ($nota ≥ 9) {
    $calificacion = "Sobresaliente";
} elseif ($nota ≥ 7) {
    $calificacion = "Notable";
} elseif ($nota ≥ 6) {
    $calificacion = "Bien";
} elseif ($nota ≥ 5) {
    $calificacion = "Suficiente";
} else {
    $calificacion = "Insuficiente";
}

echo "Un $nota es un $calificacion";
```

📄 Ejemplo: `03-elseif-notas.php`

PHP comprueba las condiciones **de arriba abajo** y, en cuanto encuentra **la primera verdadera**, ejecuta ese bloque y **se salta todas las demás**. Un 7.5 también es `≥ 6` y `≥ 5`, pero da igual: ya entró en «Notable».

Por eso **el orden importa**. Mirad qué pasa si ponemos primero la condición más general:

04-orden-importa.php — php-paso-a-paso/clase08 — Visual Studio Code

04-orden-importa.php

```php
<?php
$nota = 9.5;

// ❌ Mal: un 9.5 también es ≥ 5, así que entra aquí y no sigue mirando
if ($nota ≥ 5) {
    $mal = "Aprobado";
} elseif ($nota ≥ 9) {
    $mal = "Sobresaliente";   // nunca llega
} else {
    $mal = "Suspenso";
}
```

📄 Ejemplo: `04-orden-importa.php`



🎯 **Regla:** con condiciones del tipo «mayor o igual», id **de la más exigente a la menos exigente**.



CLASE 08 · PUNTO 5

🔍 Condiciones compuestas: &&, || y !

Muchas decisiones dependen de **varias** cosas a la vez. Para juntarlas usamos los operadores lógicos [2]:

Operador	Se lee	Es verdadero cuando…
`&&`	«y»	se cumplen **las dos** condiciones
`||`	«o»	se cumple **al menos una**
`!`	«no»	la condición es **falsa** (le da la vuelta)

05-condiciones-compuestas.php — php-paso-a-paso/clase08 — Visual Studio Code

05-condiciones-compuestas.php

```php
<?php
$edad = 17;
$conPermiso = true;
$esFinde = true;
$llueve = false;

// && : se tienen que cumplir LAS DOS
if ($edad >= 16 && $conPermiso) {
    echo "Puede ir de excursión<br>";
}

// || : basta con que se cumpla UNA
if ($esFinde || $edad < 6) {
    echo "No hay clase<br>";
}

// ! : da la vuelta a la condición
if (!$llueve) {
    echo "Salimos al patio<br>";
}

// Paréntesis para dejar claro qué va con qué
if (($esFinde || $edad < 6) && !$llueve) {
    echo "¡Al parque!";
}
```

📄 Ejemplo: `05-condiciones-compuestas.php`

La tabla de verdad

A	B	`A && B`	`A || B`	`!A`
`true`	`true`	`true`	`true`	`false`
`true`	`false`	`false`	`true`	`false`
`false`	`true`	`false`	`true`	`true`
`false`	`false`	`false`	`false`	`true`

Dos detalles que os ahorrarán disgustos:

- **Cortocircuito.** Con `&&`, si la primera condición es falsa, PHP **ni mira** la segunda (ya sabe que el resultado es falso). Con `||`, si la primera es verdadera, tampoco. Por eso podemos escribir `$b !== 0 && $a / $b > 2` sin miedo a dividir entre cero.
- **Paréntesis siempre que dudéis.** `&&` tiene más prioridad que `||`, igual que la multiplicación la tiene sobre la suma. Con paréntesis, quien lea vuestro código no tiene que acordarse.



⚠️ Existen también `and` y `or` escritos con letras, pero tienen **menos prioridad que el** `=` y provocan errores muy raros. Usad siempre `&&` y `||`.



CLASE 08 · PUNTO 6

🤔 Pensar antes de programar y no anidar

Casi nadie se atasca por no recordar cómo se escribe un `if`. La gente se atasca delante de la pantalla en blanco, sin saber por dónde empezar. Mi consejo: escribid primero la solución **en español**, como si se la explicarais a alguien. A eso se le llama **pseudocódigo**.

Problema: ¿puede una persona presentarse al examen práctico del carné?

Esquema

```
SI tiene 18 años o más
    SI ha aprobado el teórico → "Puede presentarse al práctico"
    SI NO                     → "Tiene que aprobar el teórico"
SI NO → "Todavía no tiene edad"
```

Traducirlo a PHP es casi copiar (`06-pensar-antes.php`):

ejemplo.php — php-paso-a-paso/clase08 — Visual Studio Code

ejemplo.php

```php
<?php
$edad = 19;
$teoricoAprobado = false;

if ($edad >= 18) {
    if ($teoricoAprobado) {
        $mensaje = "Puede presentarse al práctico";
    } else {
        $mensaje = "Tiene que aprobar el teórico";
    }
} else {
    $mensaje = "Todavía no tiene edad";
}
```

Funciona, pero cada `if` dentro de otro añade un nivel de sangría. Con tres o cuatro niveles, el código se convierte en una escalera imposible de leer. El truco es **descartar primero los casos que no cumplen**; lo que queda al final es el caso bueno (`07-if-plano.php`):

ejemplo.php — php-paso-a-paso/clase08 — Visual Studio Code

ejemplo.php

```php
<?php
if ($edad < 18) {
    $mensaje = "Todavía no tiene edad";
} elseif (!$teoricoAprobado) {
    $mensaje = "Tiene que aprobar el teórico";
} else {
    $mensaje = "Puede presentarse al práctico";
}
```

Mismo resultado, un solo nivel. Se lee como el portero de la discoteca: «¿Edad? No. Fuera. ¿Entrada? No. Fuera. Adelante». 😄

Mi receta en cuatro pasos

Paso	Pregunta que os tenéis que hacer
1. **Datos**	¿Qué información tengo? (edad, teórico aprobado)
2. **Resultado**	¿Qué tengo que mostrar? (uno de tres mensajes)
3. **Pseudocódigo**	¿Cómo lo resolvería yo a mano, paso a paso?
4. **Prueba**	¿Qué valores uso para comprobar **todos** los caminos?

El paso 4 es el que más se olvida. Aquí hay tres caminos, así que necesitáis al menos tres pruebas: 16 años; 19 sin teórico; 19 con teórico.



CLASE 08 · PUNTO 7

⚠️ La trampa del igual solitario

08-trampa-igual.php — php-paso-a-paso/clase08 — Visual Studio Code

🐘 08-trampa-igual.php

```php
<?php
$x = 3;

if ($x = 10) {          // ❌ NO compara: GUARDA un 10 en $x (y 10 es "verdadero")
    echo "Entra siempre, y ahora \$x vale $x<br>";
}

$y = 3;
if ($y == 10) {         // ✅ Compara
    echo "Esto no se ve";
} else {
    echo "Con == no entra, porque \$y vale $y";
}
```

📄 Ejemplo: `08-trampa-igual.php`

Con un solo `=`, PHP guarda el 10 en `$x` y, como 10 cuenta como verdadero, **siempre** entra en el `if`. No da ningún error: simplemente hace algo distinto de lo que queríais. Es de los fallos más difíciles de ver, así que revisad vuestras condiciones: dentro de un `if`, **tres iguales**.



CLASE 08 · PUNTO 8

📘 switch: la forma clásica de elegir

Cuando comparamos **una misma variable** con muchos valores posibles, una ristra de `elseif` se hace pesada. La forma clásica de resolverlo es `switch` [3]:

09-switch.php — php-paso-a-paso/clase08 — Visual Studio Code

09-switch.php

```php
<?php
$dia = "sábado";

switch ($dia) {
    case "lunes":
    case "martes":
    case "miércoles":
    case "jueves":
    case "viernes":
        $plan = "A clase";
        break;           // sin break, seguiría ejecutando el caso siguiente
    case "sábado":
        $plan = "Repasar los ejercicios";
        break;
    case "domingo":
        $plan = "Descansar";
        break;
    default:
        $plan = "Ese día no existe";
}

echo "El $dia toca: $plan";
```

📄 Ejemplo: `09-switch.php`

- Cada `case` es un valor posible. Varios `case` seguidos comparten el mismo código.
- `default` es el «en cualquier otro caso», como el `else`.
- `break` termina el `switch`. Y aquí está su punto débil: si lo olvidáis, PHP **sigue ejecutando** el caso de abajo:

10-switch-sin-break.php — php-paso-a-paso/clase08 — Visual Studio Code

10-switch-sin-break.php

```php
<?php
switch ($opcion) {
    case 1:
        $mensajes .= "Has elegido el 1. ";
        // ❌ falta el break: sigue con el caso 2
    case 2:
        $mensajes .= "Has elegido el 2. ";
        break;
}
// Con $opcion = 1: "Has elegido el 1. Has elegido el 2."
```

📄 Ejemplo: `10-switch-sin-break.php`

Además, `switch` compara con `==` (el igual débil de la clase 7), así que `"1"` y `1` cuentan como iguales. Lo vais a ver en muchísimo código, así que hay que saber leerlo. Pero para escribir código nuevo, PHP 8 nos trajo algo mejor.



`match` (PHP 8) **elige un valor** según otro valor [4]:

11-match.php — php-paso-a-paso/clase08 — Visual Studio Code

11-match.php

```php
<?php
$rol = "editor";

$permisos = match ($rol) {
    "admin" => "Control total",
    "editor", "redactor" => "Puede crear y editar contenidos",
    "alumno" => "Acceso a cursos y ejercicios",
    default => "Solo lectura",
};

$dia = 6;   // 1 = lunes … 7 = domingo
$tipoDia = match ($dia) {
    1, 2, 3, 4, 5 => "laborable",
    6, 7 => "fin de semana",
    default => "día incorrecto",
};
```

📄 Ejemplo: `11-match.php`

	`switch`	`match`
Compara con	`==` (débil): `"1"` y `1` coinciden	`===` (estricto)
`break`	Obligatorio; si se olvida, «cae» al siguiente	No existe: ejecuta un brazo y termina
Devuelve un valor	No	Sí: se guarda directamente en una variable
Si nada coincide	No hace nada	Error `UnhandledMatchError` (si no hay `default`)
Varios valores por caso	Varios `case` seguidos	Separados por comas: `6, 7 =>`

match (true): condiciones en lugar de valores

El truco más útil de `match`: si le pasamos `true`, cada brazo puede ser **una condición**, y gana la primera que sea verdadera. Es el sustituto perfecto de una cadena de `elseif` cuando solo queremos obtener un valor:

12-match-true.php — php-paso-a-paso/clase08 — Visual Studio Code

12-match-true.php

```php
<?php
$imc = 26.4;

$categoria = match (true) {
    $imc < 18.5 => "Bajo peso",
    $imc < 25 => "Peso normal",
    $imc < 30 => "Sobrepeso",
    default => "Obesidad",
};
```

📄 Ejemplo: `12-match-true.php`

Sin default, error

13-match-sin-default.php — php-paso-a-paso/clase08 — Visual Studio Code

13-match-sin-default.php

```php
<?php
$mes = 13;

$estacion = match ($mes) {
    12, 1, 2 => "invierno",
    3, 4, 5 => "primavera",
    6, 7, 8 => "verano",
    9, 10, 11 => "otoño",
};
// Error: Unhandled match case 13
```

📄 Ejemplo: `13-match-sin-default.php`

Parece una molestia, pero es una ventaja: os avisa de un caso que no habíais previsto en lugar de seguir como si nada.



⚠️ Como `match` compara con `===`, si `$dia` vale `"6"` (un texto que llega de un formulario), **no** coincide con `6`. Convertid primero: `match ((int) $dia)`.



CLASE 08 · PUNTO 10

😮 El operador ternario

Muchas veces el `if…else` solo sirve para **elegir entre dos valores**. Para eso existe el **operador ternario**, un `if…else` en una línea [5]:

Esquema

```
condición ? valor si se cumple : valor si no se cumple
```

14-ternario.php — php-paso-a-paso/clase08 — Visual Studio Code

14-ternario.php

```php
<?php
$edad = 15;
$stock = 0;
$cantidad = 1;

$tipo = $edad >= 18 ? "adulto" : "menor";
$disponible = $stock > 0 ? "En stock" : "Agotado";
// Muy útil para los plurales
$texto = $cantidad . ($cantidad === 1 ? " unidad" : " unidades");
```

📄 Ejemplo: `14-ternario.php`

Comparad con la versión larga:

15-ternario-anidado.php — php-paso-a-paso/clase08 — Visual Studio Code

15-ternario-anidado.php

```php
<?php
if ($edad >= 18) {
    $tipo = "adulto";
} else {
    $tipo = "menor";
}
```

Cinco líneas contra una. Pero cuidado con abusar: el ternario es para elegir **un valor** entre **dos**. Si hay que hacer varias cosas, o hay más de dos opciones, usad `if` o `match`.

No anidéis ternarios

15-ternario-anidado.php — php-paso-a-paso/clase08 — Visual Studio Code

15-ternario-anidado.php

```php
<?php
$nota = 6;

// ❌ $texto = $nota ⩾ 9 ? "Sobresaliente" : $nota ⩾ 5 ? "Aprobado" : "Suspenso";
//    En PHP 8 esto es un error de sintaxis

// ✅ Funciona, pero cuesta leerlo
$texto = $nota ⩾ 9 ? "Sobresaliente" : ($nota ⩾ 5 ? "Aprobado" : "Suspenso");

// ✅✅ Mucho mejor con match (true)
$mejor = match (true) {
    $nota ⩾ 9 ⇒ "Sobresaliente",
    $nota ⩾ 5 ⇒ "Aprobado",
    default ⇒ "Suspenso",
};
```

📄 Ejemplo: `15-ternario-anidado.php`

PHP 8 directamente **no deja** encadenar ternarios sin paréntesis, de tan confusos que resultaban. Mi consejo: si os veis poniendo un ternario dentro de otro, pasaos a `match (true)`.



CLASE 08 · PUNTO 11

🔍 Los operadores del PHP moderno: ?:, ??, ??= y ?→

Las versiones modernas de PHP han traído varios atajos para el caso más repetido de todos: **«usa este valor y, si no hay, este otro»**. Se parecen mucho entre sí, así que vamos uno por uno y al final os dejo una tabla para no confundirlos.

?: el ternario corto (el «operador Elvis»)

Es un ternario al que le falta la parte del medio: `$a ?: $b` significa «**si `$a` es verdadero, `$a`; si no, `$b`**». Lo llaman *Elvis* porque `?:` recuerda a su tupé. 🕺

16-elvis.php — php-paso-a-paso/clase08 — Visual Studio Code

16-elvis.php

```php
<?php
$apodo = "";
$nombre = "Lucía";
$descuento = 0;

$saludo = $apodo ?: $nombre;                    // "" es falso → usa $nombre
$textoDescuento = $descuento ?: "sin descuento";  // 0 es falso

echo "Hola, $saludo<br>";           // Hola, Lucía
echo "Descuento: $textoDescuento";  // Descuento: sin descuento
```

📄 Ejemplo: `16-elvis.php`

?? fusión de null

Ya lo visteis en la clase 7. `$a ?? $b` significa «**si** `$a` **existe y no es** `null`**,** `$a`**; si no,** `$b`». Es perfecto para los datos que pueden no llegar, como los de la URL, porque **no da aviso** aunque la clave no exista [6]:

17-fusion-null.php — php-paso-a-paso/clase08 — Visual Studio Code

17-fusion-null.php

```php
<?php
// Probad: 17-fusion-null.php?ciudad=Santander&puntos=0
$puntosPorDefecto = 10;

$ciudad = $_GET["ciudad"] ?? "sin ciudad";
$puntos = $_GET["puntos"] ?? $puntosPorDefecto;

// Encadenado: el primero que exista
$idioma = $_GET["idioma"] ?? $_GET["lang"] ?? "es";
```

📄 Ejemplo: `17-fusion-null.php`

La diferencia clave entre `?:` **y** `??` está en el `0` y en el texto vacío. Probad el ejemplo con `?puntos=0`:

- Con `??`, `$puntos` vale `0`: el dato **existe**, aunque sea un cero.
- Con `?:`, valdría `10`: el `0` cuenta como falso y lo sustituiría.

¿Cuál queréis? Depende del caso: si un `0` es un valor válido (cero puntos), `??`. Si un valor vacío debe sustituirse (un apodo en blanco), `?:`.

??= asignación con fusión de null

`$a ??= $b` significa «**si** `$a` **no existe o es** `null`**, guárdale** `$b`». Si ya tenía valor, no lo toca. Es ideal para poner valores por defecto:

18-fusion-null-asignacion.php — php-paso-a-paso/clase08 — Visual Studio Code

18-fusion-null-asignacion.php

```php
<?php
$config = [
    "tema" => "oscuro",
];

$config["tema"] ??= "claro";      // ya tenía valor: no cambia
$config["idioma"] ??= "es";       // no existía: lo crea

$contador = null;
$contador ??= 0;                  // era null: ahora vale 0
```

📄 Ejemplo: `18-fusion-null-asignacion.php`

Es la versión corta de `$config["idioma"] = $config["idioma"] ?? "es";`.

?→ el operador nullsafe

Este os lo presento para que os suene, porque trabaja con **objetos**, que veremos a partir de la clase 17. Con un objeto usamos la flecha `→` para pedirle cosas (`$fecha→format("d/m/Y")`). Pero si la variable vale `null`, la flecha da un error. El operador **nullsafe** `?→` (PHP 8) dice: «**si es `null`, no sigas y devuelve `null`**» [7]:

19-nullsafe.php — php-paso-a-paso/clase08 — Visual Studio Code

19-nullsafe.php

```php
<?php
$fechaTexto = "";   // probad también con "2026-12-25"

$fecha = $fechaTexto !== "" ? new DateTime($fechaTexto) : null;

// Sin ?→ tendríamos que preguntar antes si $fecha es null
$bonita = $fecha?->format("d/m/Y") ?? "sin fecha";

echo "Fecha: $bonita";
```

📄 Ejemplo: `19-nullsafe.php`

Fijaos en la combinación `?→` + `??`: si no hay fecha, `?→` devuelve `null`, y `??` lo cambia por «sin fecha». La retomaremos en la clase 19.

Resumen para no confundirlos

Operador	Se lee	Usa el valor de la izquierda si…	Ejemplo
`a ? b : c`	ternario	(elige `b` si `a` es verdadero; si no, `c`)	`$edad >= 18 ? "adulto" : "menor"`
`a ?: b`	Elvis	`a` es **verdadero** (no vale `0`, `""`, `null`…)	`$apodo ?: $nombre`
`a ?? b`	fusión de null	`a` **existe** y no es `null`	`$_GET["id"] ?? 0`
`a ??= b`	asignación de fusión	(guarda `b` en `a` solo si `a` no existe o es `null`)	`$config["idioma"] ??= "es"`
`a?->b`	nullsafe	(si `a` es `null`, devuelve `null` sin dar error)	`$fecha?->format("d/m/Y")`



CLASE 08 · PUNTO 12

👌 La nave espacial: `<=>`

Un operador con nombre de película. 🚀 Compara dos valores y devuelve **tres** resultados posibles: `-1` si el de la izquierda es menor, `0` si son iguales y `1` si es mayor [2]:

20-nave-espacial.php — php-paso-a-paso/clase08 — Visual Studio Code

20-nave-espacial.php

```php
<?php
$miNota = 7;
$notaDeAna = 8;

$resultado = $miNota <=> $notaDeAna;    // -1

$mensaje = match ($resultado) {
    -1 => "Ana tiene más nota",
    0 => "Empate",
    1 => "Tengo más nota que Ana",
};
```

📄 Ejemplo: `20-nave-espacial.php`

Ahora mismo parece una curiosidad, pero en la clase 11 lo usaremos para **ordenar** listas a nuestro gusto con `usort()`.



CLASE 08 · PUNTO 13

👍 Condicionales dentro del HTML

En una página real, los condicionales sirven sobre todo para decidir **qué HTML se muestra**. Con llaves `{ }` mezcladas con etiquetas uno se pierde enseguida, así que usamos la **sintaxis alternativa** (`if:` … `else:` … `endif;`), que ya visteis en la clase 5, y el **ternario** dentro de los atributos:

21-condicionales-html.php — php-paso-a-paso/clase08 — Visual Studio Code

21-condicionales-html.php

```php
<?php if ($usuarioLogueado): ?>
    <h1>Bienvenida, <?= $nombre ?></h1>
    <p class="<?= $mensajesSinLeer > 0 ? "aviso" : "normal" ?>">
        Tienes <?= $mensajesSinLeer ?> <?= $mensajesSinLeer === 1 ? "mensaje" : "mensajes" ?
> sin leer
    </p>
    <?php if ($esAdmin): ?>
        <a href="#">Panel de administración</a>
    <?php endif; ?>
<?php else: ?>
    <a href="#">Iniciar sesión</a>
<?php endif; ?>
```

📄 Ejemplo: `21-condicionales-html.php`

Cambiad `$usuarioLogueado`, `$mensajesSinLeer` y `$esAdmin` y recargad: la página se transforma. Así funcionan, por dentro, todas las webs con usuarios.



CLASE 08 · PUNTO 14

🤔 ¿Qué uso en cada caso?

Situación	Usad...
Hacer algo solo si se cumple una condición	`if`
Dos caminos con varias instrucciones cada uno	`if` / `else`
Varios caminos con condiciones distintas	`if` / `elseif` / `else`
Obtener un valor según el valor de una variable	`match`
Obtener un valor según varias condiciones	`match (true)`
Elegir un valor entre dos	ternario `? :`
Un valor por defecto si algo está vacío	`?:`
Un valor por defecto si algo no existe o es `null`	`??`
Rellenar una variable solo si no tiene valor	`??=`
Pedirle algo a un objeto que puede ser `null`	`?->`
Leer código antiguo	`switch` (sabed leerlo; para escribir, `match`)



CLASE 08 · PUNTO 15

🚀 Ejercicios

Para cada ejercicio: **pseudocódigo primero**, código después y, al final, pruebas que pasen por **todos** los caminos. Cambiad los valores de las variables al principio del archivo para probarlos.

- ☐ **01 · Entrada de cine.** Precio normal 9 €. Menores de 4 años, gratis. Menores de 14 o mayores de 65, 6 €. Los miércoles, 5 € para el resto. → `ej01-entrada-cine.php`
- ☐ **02 · Estación del año.** Con `match`, a partir del número de mes. Si el mes no existe, «mes incorrecto». → `ej02-estacion.php`
- ☐ **03 · Par, impar o cero.** Decid si un número es par, impar o cero y, con un ternario, si es positivo o negativo. → `ej03-par-impar-cero.php`
- ☐ **04 · El mayor de tres.** Con tres números, mostrad cuál es el mayor usando `if` y `&&` (sin `max()`), y avisad si está repetido. → `ej04-mayor-de-tres.php`
- ☐ **05 · Año bisiesto.** Un año es bisiesto si es divisible entre 4, salvo los divisibles entre 100 que no lo son entre 400. Probadlo con 2024, 2023, 1900, 2000 y 2100. → `ej05-bisiesto.php`
- ☐ **06 · Calculadora con switch.** Con dos números y una operación (`+`, `-`, `*`, `/`), mostrad el resultado. Controlad la división entre cero y las operaciones desconocidas. → `ej06-calculadora-`

`switch.php`

- ☐ **07 · Gastos de envío.** Tarifa según el destino con `match` (península 4,95 €; Baleares 8,95 €; Canarias, Ceuta y Melilla 14,95 €), 1 € más por cada kilo o fracción a partir de 2 kg, y envío gratis a la península en compras de 50 € o más. Pista: `ceil()` redondea hacia arriba. → `ej07-gastos-envio.php`
- ☐ **08 · ¿Contraseña segura?** Comprobad que tiene al menos 8 caracteres, alguna mayúscula y algún número, y decid **qué le falta**. Pistas: `strlen()`, `strtolower()` y `str_contains()`. → `ej08-contrasena-segura.php`
- ☐ **09 · Semáforo.** Con `match`, la acción según el color (verde, ámbar o amarillo, rojo) y un aviso de avería para cualquier otro. → `ej09-semaforo.php`
- ☐ **10 · Saludo según la hora.** Con `date("G")` y `match (true)`: «Buenos días» de 6 a 14, «Buenas tardes» de 14 a 21 y «Buenas noches» el resto. → `ej10-saludo-hora.php`
- ☐ **11 · Ternarios por todas partes.** Leed `nombre`, `apodo` y `alumnos` de la URL con `??`; saludad por el apodo y, si está vacío, por el nombre (`?:`); escribid «no hay alumnos», «hay 1 alumno» o «hay N alumnos», y «aprobado» o «suspenso» según una nota. → `ej11-ternarios.php`
- ☐ **12 · Calculadora de IMC.** Leed el peso y la altura de la URL; si falta alguno, explicad cómo usarla; si no, calculad el IMC (peso / altura²) y su categoría con `match (true)`. → `ej12-imc.php`
- ☐ **13 · Panel de usuario.** Una página HTML con la sintaxis alternativa: el rol con `match`, el número de avisos con un ternario para la clase CSS y el plural, y enlaces que solo ven ciertos roles. → `ej13-panel-usuario.php`



🏆 **Reto:** reescribid el ejercicio 06 con `match` en lugar de `switch`. ¿Cuántas líneas os ahorráis? ¿Qué pasa ahora si la operación no existe y quitáis el `default`?



CLASE 08 · PUNTO 16

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
`if` / `else` / `elseif`	Caminos distintos según condiciones; gana la primera verdadera
El orden	De la condición más exigente a la menos exigente
`&&`, `||`, `!`	Y, o, no; con paréntesis si hay dudas
Pseudocódigo	Resolvedlo en español antes de escribir PHP
Menos anidación	Descartad primero los casos que no cumplen
`=` frente a `===`	Dentro de un `if`, siempre tres iguales
`switch`	Lo clásico: compara con `==` y necesita `break`
`match`	Lo moderno: compara con `===`, devuelve un valor, error si nada coincide
Ternario	`condición ? sí : no` para elegir entre dos valores; sin anidar
`?:` / `??` / `??=`	Valor por defecto si es falso / si no existe / rellenar si falta
`?->`	Pedirle algo a un objeto que puede ser `null`
`<=>`	Compara y devuelve -1, 0 o 1

📚 Fuentes y documentación oficial

1. Manual de PHP: if, else y elseif
2. Manual de PHP: operadores de comparación
3. Manual de PHP: switch
4. Manual de PHP: match
5. Manual de PHP: el operador ternario
6. Manual de PHP: el operador de fusión de null
7. Manual de PHP: el operador nullsafe